

Name

devnm – Identifies device name.

Syntax

/etc/devnm [names]

Description

Devnm identifies the special file associated with the mounted file system where the argument *name* resides.

This command is most commonly used by */etc/rc* to construct a mount table entry for the **root** device.

Examples

Be sure to type full pathnames in this example:

/etc/devnm /usr

If **/dev/hd1** is mounted on **/usr**, this produces:

hd1 /usr

Files

*/dev/** Device names

/etc/rc XENIX startup commands

See Also

setmnt(C)



Name

df – Report number of free disk blocks.

Syntax

df [**-t**] [**-b**] [**-f**] [**-v**] [**-i**] [*filesystems*]

Description

Df prints out the number of free blocks (512 bytes each) and free inodes available for on-line file systems by examining the counts kept in the super-blocks; *filesystems* may be specified either by device name (e.g., */dev/usr*) or by mounted directory name (e.g., */usr*). If the *file-systems* argument is unspecified, the free space on all of the mounted file systems is sent to the standard output. The list of mounted file systems is given in */etc/mnttab*.

Options include:

- t** Causes total allocated block figures to be reported as well as number of free blocks.
- b** Causes total allocated bytes to be reported as well as number of free bytes.
- f** Reports only the count of the blocks in the free list (free inodes are not reported). With this option, *df* reports on raw devices.
- v** Reports percent of blocks used as well as number of blocks used and free.
- i** Reports percent of inodes and blocks used as well as the number of inodes and blocks used and free.

The **-v** and **-i** options can not be used with other *df* options.

Files

*/dev/**
/etc/mnttab

See Also

fsck(C), **mnttab**(F)

Notes

See “Notes” under *mount(C)*.

The *df* and the *df -i* commands yield different results because *df* normally uses the block device and the *-i* option uses the raw device. The inconsistency lasts until the next invocation of *sync*.

Name

`diff` – Compares two text files.

Syntax

```
diff [ -efbh ] file1 file2
```

Description

Diff tells what lines must be changed in two files to bring them into agreement. If *file1* (*file2*) is `-`, the standard input is used. If *file1* (*file2*) is a directory, then a file in that directory with the name *file2* (*file1*) is used. The normal output contains lines of these forms:

```
n1 a n3,n4
n1,n2 d n3
n1,n2 c n3,n4
```

These lines resemble *ed* commands to convert *file1* into *file2*. The numbers after the letters pertain to *file2*. In fact, by exchanging *a* for *d* and reading backward you can ascertain how to convert *file2* into *file1*. As in *ed*, identical pairs, where *n1* = *n2* or *n3* = *n4*, are abbreviated as a single number.

Following each of these lines come all the lines that are affected in the first file flagged by `<`, then all the lines that are affected in the second file flagged by `>`.

The `-b` option causes trailing spaces and tabs to be ignored and other strings of spaces to compare equally.

The `-e` option produces a script of *a*, *c*, and *d* commands for the editor *ed*, which will recreate *file2* from *file1*. The `-f` option produces a similar script, not useful with *ed*, in the opposite order. In connection with `-e`, the following shell procedure helps maintain multiple versions of a file:

```
(shift; cat $*; echo '1,$p') | ed - $1
```

This works by performing a set of editing operations on an original ancestral file. This is done by combining the sequence of *ed* scripts given as all command line arguments except the first. These scripts are presumed to have been created with *diff* in the order given on the command line. The set of editing operations is then piped as an editing script to *ed* where all editing operations are performed on the ancestral file given as the first argument on the command line. The final version of the file is then printed on the standard output. Only an ancestral file (\$1) and a chain of version-to-version *ed* scripts (\$2,\$3,...) made by *diff* need be on hand.

Except in rare circumstances, *diff* finds the smallest sufficient set of file differences.

The **-h** option does a fast, but less rigorous job. It works only when changed stretches are short and well separated, but it also works on files of unlimited length. The **-e** and **-f** options cannot be used with the **-h** option.

Files

/tmp/d????

/usr/lib/diffh for **-h**

See Also

cmp(C), comm(C), ed(C)

Diagnostics

Exit status is 0 for no differences, 1 for some differences, 2 for errors.

Notes

Editing scripts produced under the **-e** or **-f** options do not always work correctly on lines consisting of a single period (.).

Name

diff3 – Compares three files.

Syntax

diff3 [**-ex3**] *file1 file2 file3*

Description

Diff3 compares three versions of a file, and publishes disagreeing ranges of text flagged with these codes:

====	All three files differ
====1	<i>File1</i> is different
====2	<i>File2</i> is different
====3	<i>File3</i> is different

The type of change suffered in converting a given range of a given file to some other range is indicated in one of these ways:

<i>f</i> : <i>n1</i> a	Text is to be appended after line number <i>n1</i> in file <i>f</i> , where <i>f</i> = 1, 2, or 3.
<i>f</i> : <i>n1</i> , <i>n2</i> c	Text is to be changed in the range from line <i>n1</i> to line <i>n2</i> . If <i>n1</i> = <i>n2</i> , the range may be abbreviated to <i>n1</i> .

The original contents of the range follows immediately after a **c** indication. When the contents of two files are identical, the contents of the lower-numbered file is suppressed.

Under the **-e** option, *diff3* publishes a script for the editor, *ed*, that will incorporate into *file1* all changes between *file2* and *file3*, i.e., the changes that normally would be flagged with “====” and “====3”. The **-x** option produces a script to incorporate changes flagged with “====”. Similarly, the **-3** option produces a script to incorporate changes flagged with “====3”. The following command applies an editing script to *file1*:

```
(cat script; echo '1,$p') | ed -file1
```

Files

/tmp/d3*

/usr/lib/diff3prog

See Also

`diff(C)`

Notes

The `-e` option does not work properly for lines consisting of a single period.

The input file size limit is 64K bytes.

Name

`dircmp` – Compares directories.

Syntax

`dircmp [ -d ] [ -s ] dir1 dir2`

Description

Dircmp examines *dir1* and *dir2* and generates tabulated information about the contents of the directories. Listings of files that are unique to each directory are generated in addition to a list that indicates whether the files common to both directories have the same contents.

There are three options available:

- `-d` Performs a full *diff* on each pair of like-named files if the contents of the files are not identical.
- `-s` Reports whether the files are ““different”.”

`cmp(C)`, `diff(C)`.



Name

`dirname` – Delivers directory part of pathname.

Syntax

`dirname string`

Description

Dirname delivers all but the last component of the pathname in *string* and prints the result on the standard output. If there is only one component in the pathname, only a “dot” is printed. It is normally used inside substitution marks (“”) within shell procedures.

The companion command *basename* deletes any prefix ending in a slash (/) and the *suffix* (if present in *string*) from *string*, and prints the result on the standard output.

Examples

The following example sets the shell variable NAME to `/usr/src/cmd`:

```
NAME='dirname /usr/src/cmd/cat.c'
```

This example prints `/a/b/c` on the standard output:

```
dirname /a/b/c/d
```

This example prints a “dot” on the standard output:

```
dirname file.ext
```

See Also

`basename(C)`, `sh(C)`



Name

`disable` – Turns off terminals and printers.

Syntax

```
disable tty ...
disable [-c] [-r [reason]] printers
```

Description

For terminals, this program manipulates the `/etc/ttys` file and signals `init` to disallow logins on a particular terminal. For printers, `disable` stops print requests from being sent to the named printer. The following options can be used:

- `-c` Cancels any requests that are currently printing.
- `-r[reason]` Associates a *reason* with disabling the printer. The *reason* applies to all printers listed up to the next `-r` option. If *reason* is more than one word, you must enclose it within quotes. If the `-r` option is not present, or is given without a *reason*, then the default *reason* (“reason unknown”) is used. *Reason* is reported by `lpstat(C)`.

Examples

In this example, a printer named *linepr* is disabled because of a paper jam:

```
disable -r “paper jam” linepr
```

Files

```
/dev/tty*
/etc/ttys
/usr/spool/lp/*
```

See Also

`enable(C)`, `getty(M)`, `init(M)`, `login(M)`, `lp(C)`, `lpinit(C)`, `lpstat(C)`, `ttys(M)`

Warning

Be absolutely certain to pause at least *one minute* before reusing this command or before using the *enable* command. Failure to do so may cause the system to crash.

Bugs

The *disable* command cancels the current job even without the *-c* option. You may wish to change *disable* to be executable only by root. To do this, type (as root) the following commands and press RETURN after each:

```
cd /bin
rm disable
cp enable disable
chmod 700 disable
```

Name

`dmesg` – Displays the system messages on the console.

Syntax

`dmesg [ - ]`

Description

The *dmesg* command displays all the system messages that have been generated since the last time the system was booted. If the “-” option is specified, it displays only those messages that have been generated since the last time the *dmesg* command was executed.

Dmesg is periodically invoked according to the instructions in the file `/usr/spool/cron/crontabs/root`. It is also automatically invoked by the `/etc/rc` file whenever the system is booted.

A history of all system messages is recorded in the file `/usr/adm/messages`. This file will continue to grow unless the user occasionally erases its contents.

Files

`/etc/dmesg`
`/etc/rc`
`/usr/adm/messages`
`/usr/adm/msgbuf`
`/usr/spool/cron/crontabs/root`

See Also

`cron(C)`

Name

`dos` – `doscat` `doscpc` `dosdir` `dosls` `dosmkdir` `dosrm` `dosrmdir` –DOS Utilities

Syntax

`doscat` [`-r`] *file* ...

`doscpc` [`-r`] *file1 file2*

`doscpc` [`-r`] *file* ... *directory*

`dosdir` *directory* ...

`dosls` *directory* ...

`dosmkdir` *directory* ...

`dosrm` *file* ...

`dosrmdir` *directory* ...

Description

The *dos* commands provide access to the files and directories on MS-DOS disks. The commands perform the following actions:

doscat Copies one or more MS-DOS files to the standard output. If `-r` is given, the files are copied without NEWLINE conversions (see “Conversions” below).

doscpc Copies files between an MS-DOS disk and a XENIX file system. If *file1* and *file2* are given, *file1* is copied to *file2*. If a *directory* is given, one or more *files* are copied to that directory. If the `-r` option is given, the files are copied without NEWLINE conversions (see “Conversions” below).

dosdir Lists MS-DOS files in the standard MS-DOS style directory format.

dosls Lists MS-DOS directories and files in a XENIX style (see *ls(C)*).

dosrm Removes files from an MS-DOS disk.

dosmkdir Creates a directory on an MS-DOS disk.

dosrmdir Deletes directories from an MS-DOS disk.

The *file* and *directory* arguments for MS-DOS files and directories have the form:

device:name

where *device* is a XENIX pathname for the special device file containing the MS-DOS disk, and *name* is a pathname to a file or directory on the MS-DOS disk. The two components are separated by a colon (:). For example, the argument:

`/dev/fd0:/src/file.asm`

specifies the MS-DOS file, **file.asm**, in the **/src** directory on the disk in the device file, **/dev/fd0**. Note that slashes (and not backslashes) are used as filename separators for MS-DOS pathnames. Arguments without a *device:* are assumed to be XENIX files.

For convenience, the default file, **/etc/default/dos**, can define MS-DOS drive names to be used in place of the special device file pathnames. For example, if the file contains the following line:

`C=/dev/hd0d`

then drive letter “A” may be used in place of the special device file pathname, **/dev/hd0d**, when referencing MS-DOS files (see “Examples” below).

The commands operate on most MS-DOS formatted floppy diskettes. To operate on a DOS hard disk partition, the architecture of the hard disk must include a bad-track table and a partition table.

Conversions

All MS-DOS text files use a RETURN-LINEFEED combination, CR-LF, to indicate a NEWLINE. XENIX uses a single NEWLINE LF character. When the *doscat* and *doscp* commands transfer MS-DOS text files to XENIX, they automatically strip the CR. When text files are transferred to MS-DOS, the commands insert the CR before each LF character. The **-r** option can be used to override the automatic conversion and force the command to perform a true byte copy regardless of file type.

Examples

```
doscat /dev/fd0:/docs/memo.txt
doscat /tmp/f1 /tmp/f2 /dev/fd0:/src/file.asm
```

```
dosdir /dev/fd0:/src
dosdir A:/src A:/dev
```

```
doscp /tmp/myfile.txt /dev/fd0:/docs/memo.txt
doscp /tmp/f1 /tmp/f2 /dev/fd0:/mydir
```

```
dosls /dev/fd0:/src
dosls B:
```



```
dosmkdir /dev/fd0:/usr/docs
```

```
dosrm /dev/fd0:/docs/memo.txt  
dosrm A:/docs/memo1.txt
```

```
dosrmdir /dev/fd0:/usr/docs
```

Files

/etc/default/dos Default information

/dev/fd* Floppy disk devices

/dev/hd* Hard disk devices

See Also

assign(C), default(M), dtype(C)

Notes



It is not possible to refer to MS-DOS directories with wildcard specifications. The programs mentioned above cooperate among themselves so no two programs will access the same MS-DOS disk. Only one process will access a given MS-DOS disk at any time, while other processes wait. If a process has to wait too long, it will print the error message, “can’t seize a device”, and will exit with an exit code of 1.



Name

`dtype` – Determines disk type.

Syntax

`dtype [-s] device ...`

Description

Dtype determines type of disk, prints pertinent information on the standard output unless the silent (`-s`) option is selected, and exits with a corresponding code (see below). When more than one argument is given, the exit code corresponds to the last argument.

Disk Type	Exit Code	Message (optional)
Misc.	60	error (specified)
	61	empty or unrecognized data
Storage	70	dump format, volume n
	71	tar format[, extent e of n]
	72	cpio format
	73	cpio character (<code>-c</code>) format
MS-DOS	80	MS-DOS 1.x, 8 sec/track, single sided
	81	MS-DOS 1.x, 8 sec/track, dual sided
	90	MS-DOS 8 sec/track, single sided
	91	MS-DOS 8 sec/track, dual sided
	92	MS-DOS 9 sec/track, single sided
	93	MS-DOS 9 sec/track, dual sided
	94	MS-DOS fixed disk
XENIX	110	MS-DOS 9 or 15 sec/track, dual sided
	120	XENIX 2.x filesystem[needs fsck]
	130	XENIX 3.x/5.x filesystem[needs fsck]

Notes

XENIX file systems and *dump* and *cpio* binary formats may not be recognized if created on a foreign system. This is due to such system differences as byte and word swapping and structure alignment.

Bugs

The *dtype* command has problems with some dump volume formats.



Name

du – Summarizes disk usage.

Syntax

du [**-afrsu**] [*names*]

Description

Du gives (recursively) the number of blocks contained in all files and directories within each directory and file specified by the *names* argument. This command assumes a block size of 512 bytes. The block count includes the indirect blocks of the file. If *names* is missing, the current directory is used.

The optional **-s** argument causes only the grand total (for each of the specified *names*) to be given. The optional **-a** argument causes an entry to be generated for each file. Absence of either causes an entry to be generated for each directory only.

The **-f** option causes *du* to display the usage of files in the current file system only. Directories containing mounted file systems will be ignored. The **-u** option causes *du* to ignore files that have more than one link.

Du is normally silent about directories that cannot be read, files that cannot be opened, etc. The **-r** option will cause *du* to generate messages in such instances.

A file with two or more links is only counted once.

Notes

If the **-a** option is not used, nondirectories given as arguments are not listed.

If there are too many distinct linked files, *du* will count the excess files more than once.

Files with holes in them will get an incorrect block count.

Bugs

When *du* is used on large, deeply nested directories (approximately 10 or more levels), it may silently fail to traverse some branches of the directory. The **-r** option will cause *du* to report such failures.



Name

dumpdir – Prints the names of files on a backup archive.

Syntax

dumpdir [*f filename*]

Description

Dumpdir is used to list the names and inode numbers of all files and directories on an archive written with the *backup* command. This is most useful when attempting to determine the location of a particular file in a set of backup archives.

The *f* option causes *filename* to be used as the name of the backup device instead of the default. The backup device depends on the setting of the variable *TAPE* in the */etc/default/dumpdir* file.

Files

rst* Temporary files

See Also

backup(C), **default(M)**, **restore(C)**

Diagnostics

If the backup extends over more than one volume (where a likely volume is a floppy disk or tape), you will be asked to change volumes. Press RETURN after changing volumes.



Name

echo – Echoes arguments.

Syntax

echo [**-n**] [**-u**] [**-e**] [*arg*]...

Description

Echo writes its arguments separated by blanks and terminated by a NEWLINE on the standard output. The *echo* options are:

-n

Prints line without NEWLINE; same as using `\c` escape sequence.

-u

Prevents output from being buffered.

-e

Writes to standard error instead of standard output.

Echo also understands C-like escape conventions. The following escape sequences need to be quoted so that the shell interprets them correctly:

`\b` BACKSPACE

`\c` Prints line without NEWLINE; same as use of **-n** option

`\f` FORMFEED

`\n` NEWLINE

`\r` RETURN

`\t` TAB

`\v` Vertical TAB

`\\` Backslash

`\n` The 8-bit character whose ASCII code is the 1, 2 or 3-digit octal number *n*, which must start with a zero

Echo is useful for producing diagnostics in command files and for sending known data into a pipe.

See Also

sh(C)



Name

`ed` – Invokes the text editor.

Syntax

`ed [ - ] [ -p string ] [ file ]`

Description

Ed is the standard text editor. If the *file* argument is given, *ed* simulates an *e* command (see below) on the named file; that is, the file is read into *ed*'s buffer so that it can be edited. *Ed* makes changes to a copy of the file it is editing; changes made to the copy have no effect on the file until a *w* (write) command is given. The copy of the text being edited resides in a temporary file called the *buffer*. There is only one buffer.

The options are:

- Suppresses the printing of character counts by the *e*, *r*, and *w* commands, diagnostics from the *e* and *q* commands, and the appearance of the *!* prompt after a *!shell command*.
- p Allows the user to specify a prompt string.

Ed supports formatting capability. After including a format specification as the first line of *file* and invoking *ed* with your terminal in `stty -tabs` or `stty tab3` mode (see *stty*(C)), the specified tab stops will be used automatically when scanning *file*. For example, if the first line of a file contained:

```
<:t5,10,15 s72:>
```

tab stops would be set at columns 5, 10, and 15, and a maximum line length of 72 would be imposed. NOTE: While inputting text, TAB characters when typed are expanded to every eighth column by default.

Commands to *ed* have a simple and regular structure: zero, one, or two *addresses* followed by a single-character *command*, possibly followed by parameters to that command. These addresses specify one or more lines in the buffer. Every command that requires addresses has default addresses, so that the addresses can often be omitted.

In general, only one command may appear on a line. Certain commands allow the input of text. This text is placed in the appropriate place in the buffer. While *ed* is accepting text, it is said to be in *input mode*. In this mode, *no* commands are recognized; all input is merely collected. Input mode is left by typing a period (.) alone at the beginning of a line.

Ed supports a limited form of *regular expression* notation; regular expressions are used in addresses to specify lines and in some commands (e.g., *s*) to specify portions of a line that are to be substituted. A regular expression specifies a set of character strings. A member of this set of strings is said to be *matched* by the regular expression. The regular expressions allowed by *ed* are constructed as follows:

The following one-character regular expressions match a *single* character:

- 1.1 An ordinary character (*not* one of those discussed in 1.2 below) is a one-character regular expression that matches itself.
- 1.2 A backslash (\) followed by any special character is a one-character regular expression that matches the special character itself. The special characters are:
 - a. ., *, [, and \ (dot, star, left square bracket, and backslash, respectively), which are always special, *except* when they appear within square brackets ([]); see 1.4).
 - b. ^ (caret), which is special at the *beginning* of an *entire* regular expression (see 3.1 and 3.2), or when it immediately follows the left of a pair of square brackets ([]) (see 1.4).
 - c. \$ (dollar sign), which is special at the *end* of an *entire* regular expression (see 3.2).
 - d. The character used to bound (i.e., delimit) an entire regular expression, which is special for that regular expression (for example, see how slash (/) is used in the *g* command.)
- 1.3 A period (.) is a one-character regular expression that matches any character except NEWLINE.
- 1.4 A nonempty string of characters enclosed in square brackets ([]) is a one-character regular expression that matches *any single* character in that string. If, however, the first character of the string is a caret (^), the one-character regular expression matches any character *except* NEWLINE and the remaining characters in the string. The star (*) has this special meaning *only* if it occurs first in the string. The dash (–) may be used to indicate a range of consecutive ASCII characters; for example, [0–9] is equivalent to [0123456789]. The dash (–) loses this special meaning if it occurs first (after an initial caret (^), if any) or last in the string. The right square bracket (]) does not terminate such a string when it is the first character within it (after an initial caret (^), if any); e.g., “[ja–f]” matches either a right square bracket (]) or one of the letters “a” through “f”. Dot, star, left bracket, and the backslash lose their special meaning within such a string of characters.

The following rules may be used to construct regular expressions from one-character regular expressions:

- 2.1 A one-character regular expression matches itself.
- 2.2 A one-character regular expression followed by a star (*) is a regular expression that matches zero or more occurrences of the one-character regular expression. If there is any choice, the longest leftmost string that permits a match is chosen.
- 2.3 A one-character regular expression followed by $\{m\}$, $\{m,\}$, or $\{m,n\}$ is a regular expression that matches a *range* of occurrences of the one-character regular expression. The values of m and n must be nonnegative integers less than 255; $\{m\}$ matches *exactly* m occurrences; $\{m,\}$ matches *at least* m occurrences; $\{m,n\}$ matches any number of occurrences between m and n , inclusive. Whenever a choice exists, the regular expression matches as many occurrences as possible.
- 2.4 A concatenation of regular expressions is a regular expression that matches the concatenation of the strings matched by each component of the regular expression.
- 2.5 A regular expression enclosed between the character sequences $\langle$ and $\rangle$ is a regular expression that matches whatever the original regular expression matches. See 2.6 for a discussion of why this is useful.
- 2.6 The $\langle n \rangle$ expression matches the same string of characters that was matched by an expression enclosed between $\langle$ and $\rangle$ *earlier* in the same regular expression. Here n is a digit; the subexpression specified is that expression beginning with the n th occurrence of $\langle$ (counting from the left. For example, the expression “ $\langle \langle .*\rangle \rangle 1 \$$ ” matches a line consisting of two repeated appearances of the same string.

Finally, an *entire regular expression* may be constrained to match only an initial segment or final segment of a line (or both):

- 3.1 A caret (^) at the beginning of an entire regular expression constrains that regular expression to match an *initial* segment of a line.
- 3.2 A dollar sign (\$) at the end of an entire regular expression constrains that regular expression to match a *final* segment of a line. The construction *^entire regular expression\$* constrains the entire regular expression to match the entire line.

The null regular expression (e.g., $\langle \rangle$) is equivalent to the last regular expression encountered.

To understand addressing in *ed* it is necessary to know that there is a *current line* at all times. Generally speaking, the current line is the last line affected by a command; the exact effect on the current line is discussed under the description of each command. *Addresses* are constructed as follows:

1. The `.` character addresses the current line.
2. The `$` character addresses the last line of the buffer.
3. A decimal number, n , addresses the n th line of the buffer.
4. `'x` addresses the line marked with the mark name character x , which must be a lowercase letter. Lines are marked with the `k` command described below.
5. A regular expression enclosed by slashes (`/`) addresses the first line found by searching *forward* from the line *following* the current line toward the end of the buffer and stopping at the first line containing a string matching the regular expression. If necessary, the search wraps around to the beginning of the buffer and continues up to and including the current line, so that the entire buffer is searched.
6. A regular expression enclosed in question marks (`?`) addresses the first line found by searching *backward* from the line *preceding* the current line toward the beginning of the buffer and stopping at the first line containing a string matching the regular expression. If necessary, the search wraps around to the end of the buffer and continues up to and including the current line. See also the last paragraph before *Files* below.
7. An address followed by a plus sign (`+`) or a minus sign (`-`) and a decimal number specifies that address plus or minus the indicated number of lines. The plus sign may be omitted.
8. If an address begins with `+` or `-`, the addition or subtraction is taken with respect to the current line; e.g., `-5` is understood to mean `-.5`.
9. If an address ends with `+` or `-`, then, depending on the sign, 1 is added to or subtracted from the address. As a consequence of this rule and of Rule 8, the address refers to the line preceding the current line. (To maintain compatibility with earlier versions of the editor, the `^` character in addresses is entirely equivalent to `-`.) Also, trailing `+` and `-` characters have a cumulative effect, so `---` refers to the current line minus 2.
10. For convenience, a comma (`,`) stands for the address pair `1,$`, while a semicolon (`;`) stands for the pair `.,$`.

Commands may require zero, one, or two addresses. Commands that do not require an address regard the presence of an address as an error. Commands that accept one or two addresses assume default addresses when an insufficient number of addresses is given; if more addresses are given than such a command requires, the last address is used.

Typically, addresses are separated from each other by a comma (`,`). They may also be separated by a semicolon (`;`). In the latter case, the current line (`.`) is set to the first address, and only then is the second address calculated. This feature can be used to determine the starting line for forward and backward searches (see Rules 5 and 6). The second address of any two-address sequence must correspond to a line in the buffer that follows the line corresponding to the first address.

In the following list of *ed* commands, the default addresses are shown in parentheses. The parentheses are *not* part of the address; they show that the given addresses are the defaults.

It is generally illegal for more than one command to appear on a line. However, any command (except *e*, *f*, *r*, or *w*) may be suffixed by *p* or by *l*, in which case the current line is either printed or listed as discussed below under the *p* and *l* commands.

(.)a
<text>

The append command, *a*, reads the given text and appends it after the addressed line; the dot is left at the last inserted line, or, if there were no inserted lines, it is left at the addressed line. Address 0 is legal for this command, causing the “appended” text to be placed at the beginning of the buffer.

(.)c
<text>

The change command, *c*, deletes the addressed lines, then accepts input text that replaces these lines; the dot is left at the last line input, or, if there was no input, it is left at the first line that was not deleted.

(.,.)d

The delete command, *d*, deletes the addressed lines from the buffer. The line after the last line deleted becomes the current line; if the lines deleted were originally at the end of the buffer, the new last line becomes the current line.

e file

The edit command, *e*, deletes the entire contents of the buffer, and then reads in the named file; a dot is set to the last line of the buffer. If no filename is given, the currently remembered filename, if any, is used (see the *f* command). The number of characters read is typed and *file* is remembered for possible use as a default filename in subsequent *e*, *r*, and *w* commands. If *file* begins with an exclamation (!), the rest of the line is taken to be a shell command. The output of this command is read for the *e* and *r* commands. For the *w* command, the file is used as the standard input for the specified command. Such a shell command is *not* remembered as the current filename.

E file

The Edit command, *E*, is like *e*, except the editor does not check to see if any changes have been made to the buffer since the last *w* command.

f file

If *file* is given, the filename command changes, *f*, the currently remembered filename to *file*; otherwise, it prints the currently remembered filename.

(1,\$)g/regular-expression/command list

In the global command, *g*, the first step is to mark every line that matches the given regular expression. Then, for every such line, the given *command list* is executed with a *.*, initially set to that line. A single command or the first of a list of commands appears on the same line as the global command. All lines of a

multiline list, except the last line, must end with a \; *a*, *i*, and *c* commands and associated input are permitted; also, the *.* terminating input mode may be omitted if it would be the last line of the *command list*. An empty *command list* is equivalent to the *p* command. The *g*, *G*, *v*, and *V* commands are *not* permitted in the *command list*. See also “Notes” and the last paragraph before “Files” below.

(1,\$)G/*regular-expression*/

The interactive Global command, *G*, marks every line that matches the given regular expression. Then, for every match, the line is printed, a dot (.) is charged to that line, and any *one* command (other than one of the *a*, *c*, *i*, *g*, *G*, *v*, and *V* commands) may be input and executed. After the execution of that command, the next marked line is printed, and so on; a NEWLINE acts as a null command; an ampersand (&) causes the re-execution of the most recent command executed within the current invocation of *G*. Note that the commands input as part of the execution of the *G* command may address and affect *any* lines in the buffer. The *G* command can be terminated by typing an INTERRUPT.

h

The help command, *h*, gives a short error message that explains the reason for the most recent ? diagnostic.

H

The Help command, *H*, causes *ed* to print error messages for all subsequent ? diagnostics. It also explains the previous diagnostic if there was one. The *H* command alternately turns this mode on and off; it is initially off.

(.)i

<*text*>

.

The insert command, *i*, inserts the given text before the addressed line; a dot is left at the last inserted line, or if there were no inserted lines, it is left at the addressed line. This command differs from the *a* command only in the placement of the input text. Address 0 is not legal for this command.

(.,+1)j

The join command, *j*, joins contiguous lines by removing the appropriate NEWLINE characters. If only one address is given, this command does nothing.

(.)kx

The mark command, *k*, marks the addressed line with the name *x*, a lowercase letter. The address 'x then addresses this line; the dot is unchanged.

(.,)l

The list command, *l*, prints addressed lines. Nonprinting characters (e.g., TAB, BACKSPACE) are represented by mnemonic overstrikes; all other nonprinting characters are printed in octal, and long lines are folded. An *l* command may be appended to any command except *e*, *f*, *r*, or *w*.

(.,.)*ma*

The move command, *m*, repositions an addressed line, or lines, after the line addressed by *a*. Address 0 is legal for *a* and moves the addressed line(s) to the beginning of the file; it is an error if address *a* falls within the range of moved lines; a dot is left at the last line moved.

(.,.)*n*

The number command, *n*, prints the addressed lines, preceding each line by its line number and a TAB character; a dot is left at the last line printed. The *n* command may be appended to any command except *e*, *f*, *r*, or *w*.

(.,.)*p*

The print command, *p*, prints the addressed lines; dot is left at the last line printed. The *p* command may be appended to any command except *e*, *f*, *r*, or *w*; for example, *dp* deletes the current line and prints the new current line.

P

The editor will prompt with a * for all subsequent commands. The *P* command alternately turns this mode on and off; it is initially off.

q

The quit command, *q*, causes *ed* to exit. No automatic write of a file is done.

Q

The editor exits without checking if changes have been made in the buffer since the last *w* command.

(\$)*r file*

The read command, *r*, reads in the given file after the addressed line. If no filename is given, the currently remembered filename, if any, is used (see *e* and *f* commands). The currently remembered filename is *not* changed unless *file* is the very first filename mentioned since *ed* was invoked. Address 0 is legal for *r* and reads the file at the beginning of the buffer. If the read is successful, the number of characters read is typed, and a dot is set to the last line read in. If *file* begins with *!*, the rest of the line is taken to be a shell (*sh*(C)) command whose output is to be read. Such a shell command is *not* remembered as the current filename.

(.,.)*s/regular-expression/replacement/* or

(.,.)*s/regular-expression/replacement/g* or

(.,.)*s/regular-expression/replacement/n* *n*=1-512

The substitute command, *s*, searches each addressed line for an occurrence of the specified regular expression. In each line in which a match is found, all (nonoverlapped) matched strings are replaced by the *replacement* if the global replacement indicator *g* appears after the command. If the global indicator does not appear, only the first occurrence of the matched string is replaced. It is an error for the substitution to fail on *all* addressed lines. Any character other than SPACEBAR or NEWLINE may be used instead of */* to delimit the regular expression and the *replacement*; a dot is left at the last line on which a substitution

occurred.

An ampersand (&) appearing in the *replacement* is replaced by the string matching the regular expression on the current line. The special meaning of the ampersand in this context may be suppressed by preceding it with a backslash. The character `\n`, where *n* is a digit, is replaced by the text matched by the *n*th regular subexpression of the specified regular expression enclosed between `(` and `)`. For nested subexpressions in parentheses, *n* is determined by counting occurrences of `(` starting from the left. When `%` is the only character in the *replacement*, the *replacement* used in the most recent substitute command is used in the current substitute command. The `%` loses its special meaning when it is in a replacement string of more than one character or when it is preceded by a `\`.

A line may be split by substituting a NEWLINE character into it. The NEWLINE in the *replacement* must be escaped by preceding it with a `\`. Such a substitution cannot be done as part of a *g* or *v* command list.

(.,.)*ta*

This command acts just like the *m* command, except that a *copy* of the addressed lines is placed after address *a* (which may be 0); a dot is left at the last line of the copy.

u

The undo command, *u*, nullifies the effect of the most recent command that modified anything in the buffer, namely the most recent *a*, *c*, *d*, *g*, *i*, *j*, *m*, *r*, *s*, *t*, *v*, *G*, or *V* command.

(1,\$)*v*/regular-expression/command list

This command is the same as the global command, *g*, except that the *command list* is executed with dot initially set to every line that does *not* match the regular expression.

(1,\$)*V*/regular-expression/

This command is the same as the interactive global command, *G*, except that the lines marked during the first step do *not* match the regular expression.

(1,\$)*w* file

The write command, *w*, writes the addressed lines into the named file. If the file does not exist, it is created with mode 666 (readable and writeable by everyone), unless the *umask* setting (see *sh*(C)) dictates otherwise. The currently remembered filename is *not* changed unless *file* is the first filename mentioned since *ed* was invoked. If no filename is given, the currently remembered filename, if any, is used (see *e* and *f* commands); the dot is unchanged. If the command is successful, the number of characters written is displayed. If *file* begins with an exclamation (!), the rest of the line is taken to be a shell command to which the addressed lines are supplied as the standard input. Such a shell command is *not* remembered as the current filename.

(\$)=

The line number of the addressed line is typed; the dot is unchanged by this command.

!shell command

The remainder of the line after the **!** is sent to the XENIX shell (*sh*(C)) as a command. Within the text of that command, the unescaped **%** character is replaced with the remembered filename; if an **!** appears as the first character of the shell command, it is replaced with the text of the previous shell command. Thus, **!!** will repeat the last shell command. If any expansion is performed, the expanded line is echoed, and the dot is unchanged.

(.+1)

A lone address on a line causes that line to be printed. A lone RETURN on a line is equivalent to **+.1p**. This is useful for stepping forward through the editing buffer one line at a time.

If an interrupt signal (ASCII DEL or BREAK) is typed, *ed* prints a question mark (?) and returns to its command level.

Some size limitations are: 512 characters per line, 256 characters per global command list, 64 characters per filename, and 128K characters in the buffer. The limit on the number of lines depends on the amount of user memory.

When reading a file, *ed* discards ASCII NUL characters and all characters after the last NEWLINE. Files (e.g., **a.out**) containing characters not in the ASCII set (bit 8 on) cannot be edited by *ed*.

If the closing delimiter of a regular expression or replacement string (e.g., **/**) would be the last character before a NEWLINE, that delimiter may be omitted, in which case the addressed line is printed. Thus the following pairs of commands are equivalent:

```
s/s1/s2s/s1/s2/p
g/s1g/s1/p
?s1?s1?
```

Files

/tmp/e#	Temporary; # is the process number
ed.hup	Work is saved here if the terminal is hung up

See Also

grep(C), *regex*(S), *sed*(C), *sh*(C), *stty*(C)

Diagnostics

? Command errors
? *file* An inaccessible file

Use the help, *h*, and Help, *H*, commands for detailed explanations.

If you have made changes in the buffer since the last *w* command, *ed* warns you if you attempt to destroy it's buffer using the *e* or *q* commands: it prints ? and allows you to continue editing. A second *e* or *q* command at this point will take effect. The dash (-) command-line option inhibits this feature.

Notes

An exclamation (!) command is not subject to a *a* or a *v* command.

The ! command and the ! escape from the *e*, *r*, and *w* commands cannot be used if the the editor is invoked from a restricted shell (see *sh*(C)).

The \n sequence in a regular expression does not match any character.

The *l* command mishandles DEL.

Because 0 is an illegal address for the *w* command, you cannot create an empty file with *ed*.

Characters are mashed to 7 bits on input.

If the editor input is coming from a command file (i.e., *ed file < ed-cmd-file*), the editor will exit at the first failure of a command in the command file.

Name

enable – Turns on terminals and lineprinters.

Syntax

```
enable [ -d ] [ -e ] tty ...  
enable printers
```

Description

For terminals this program manipulates the */etc/ttys* file and signals *init* to allow logins on a particular terminal. The **-e** and **-d** options may be used to allow logins on some terminals and refuse logins on other terminals in a single command.

For lineprinters, *enable* activates the named printers and enables them to print requests taken by *lp*(C). Use *lpstat*(C) to find the status of the printers.

Examples

A simple command to enable **tty01** follows:

```
enable tty01
```

Multiple terminals can be disabled and/or enabled using the **-d** and **-e** options before the appropriate terminal names:

```
enable tty01 -e tty02 -d tty03 tty04
```

The above command line enables **tty01** and **tty02**, and disables **tty03** and **tty04**.

Files

*/dev/tty**

/etc/ttys

*/usr/spool/lp/**

See Also

disable(C), *getty*(M), *init*(M), *login*(M), *lp*(C), *lpstat*(C), *ttys*(M)

Warning

Be absolutely certain to pause at least *one minute* before reusing this command or before using the *disable* command on terminals. Failure to do so may cause the system to crash.

Name

`env` – Sets environment for command execution.

Syntax

`env [-] [ name=value ] ... [ command args ]`

Description

Env obtains the current *environment*, modifies it according to its arguments, then executes the command with the modified environment. Arguments of the form *name=value* are merged into the inherited environment before the command is executed. The `-` flag causes the inherited environment to be ignored completely, so that the command is executed with exactly the environment specified by the arguments.

If no command is specified, the resulting environment is printed, one name-value pair per line.

See Also

`environ(M)`, `exec(S)`, `profile(M)`, `sh(C)`



Name

ex – Invokes a text editor.

Syntax

ex [-] [-v] [-tag] [-r] [+lineno] *name* ...

Description

Ex is the root of the editors *ex* and *vi*. *Ex* is a superset of *ed*, whose most notable extension is a display editing facility. *Vi* focuses on display based editing.

If you have not used *ed*, or if you are a casual user, you will find that *edit* is quite convenient. It avoids some of the complexities of *ex*, which is used mostly by systems programmers and persons familiar with *ed*.

If you have a CRT terminal, you may wish to use a display based editor; in this case see *vi*(C), a command which focuses on the display editing portion of *ex*.

For ed Users

If you have used *ed*, you will find that *ex* has a number of new features. Intelligent terminals and high-speed terminals are easy to use with *vi*. Generally, the *ex* editor uses far more of the capabilities of a terminal than does *ed*. It uses the terminal capability database *termcap*(M) and the type of the terminal you are using to determine how to drive your terminal efficiently. The *ex* editor makes use of features such as the INSERT and DELETE characters and line in its *visual* command mode, which can be abbreviated *vi*, and which is the central mode of editing when using *vi*(C). There is also an interline editing *open* command, (*o*), that works on all terminals.

Ex contains a number of features for easily viewing the text of a file. The *z* command gives easy access to windows of text. Typing CTRL-D causes the editor to scroll a half-window of text and is more useful for quickly stepping through a file than repeatedly pressing the RETURN key. Of course, the screen-oriented *visual* mode gives constant access to editing context.

Ex gives you more help when you make mistakes. For example, the *undo* command (*u*) allows you to reverse any single change. *Ex* gives you a lot of feedback, by printing changed lines, and indicating when more than a few lines are affected by a command, so it is easy to detect when a command has changed more lines than it should have.

The editor also prevents overwriting of existing files unless you have edited them, so that you don't accidentally erase a file other than the one you are editing. If the system (or editor) crashes, or you accidentally hang up the phone, you can use the *recover* command to retrieve your work. This will get you back to within a few lines of where you left off.

Ex has several features for editing more than one file at a time. For example, you can give it a list of files on the command line and use the *next* command (*n*) to edit each in turn. You can also give the *next* command a list of filenames, or a pattern used by the shell to specify a new set of files to be edited. In general, filenames in the editor may be formed with full shell metasyntax. The % metacharacter is also available for forming filenames and is replaced by the name of the current file. For editing large groups of related files you can use *ex*'s tag command to locate functions and other important points in any of the files. This is useful when you want to find the definition of a particular function in a large program. The command *ctags*(CP) builds a *tags* file or a group of C programs.

For moving text between files or within a file, the editor has a group of buffers named *a* through *z*. You can place text in these buffers and carry it over when you edit another file.

The *&* command repeats the last *substitute* command. There is also a confirmed substitute command. You give a range of substitutions to be done and the editor interactively asks whether each substitution is desired.

You can use the *substitute* command in *ex* to convert the case of letters between uppercase and lowercase. It is possible to ignore case in searches and substitutions. *Ex* also allows you to construct regular expressions that match words. This is convenient, for example, when searching for the word "edit" if your document also contains the word "editor."

Ex has a set of *options* that you can set. One useful option is *autoindent* which allows the editor to supply leading white space to align text. You can then use the CTRL-D key to BACKTAB, SPACEBAR, and TAB forward to align new code easily.

Miscellaneous new useful features include an intelligent *join* (*j*) command which supplies: white space between joined lines automatically; the < and > commands which shift groups of lines; and the ability to filter portions of the buffer through commands such as *sort*.

Files

/usr/lib/ex3.7strings	Error messages
/usr/lib/ex3.7recover	Recover command
/usr/lib/ex3.7preserve	Preserve command
/etc/termcap	Describes capabilities of terminals
\$HOME/.exrc	Editor startup file
/tmp/Exnnnnnn	Editor temporary
/tmp/Rxnnnnnn	Named buffer temporary
/usr/preserve	Preservation directory

See Also

`awk(C)`, `ctags(CP)`, `ed(C)`, `grep(C)`, `sed(C)`, `termcap(M)`, `vi(C)`

Notes

The *undo* command causes all marks to be lost on lines changed and then restored if the marked lines were changed.

Undo never clears the buffer modified condition.

The *z* command prints a number of logical lines rather than physical lines. More than a screen full of output may result if long lines are present.

File input/output errors don't print a name if the command line "--" option is used.

There is no easy way to do a single scan that ignores case.

Because of the implementation of the arguments to *next*, the argument *list* cannot exceed 512 bytes.

The format of */etc/termcap* and the large number of capabilities of terminals used by the editor cause terminal type setup to be rather slow.

When exiting, remember that the editor will not warn you if you haven't used text that you have placed in any of the named buffers.

Null characters are discarded in input files and cannot appear in resultant files.

This utility was developed at the University of California at Berkeley and is used with permission.



Name

expr – Evaluates arguments as an expression.

Syntax

expr arguments

Description

The arguments are taken as an expression. After evaluation, the result is written on the standard output. Terms of the expression must be separated by blanks. Characters special to the shell must be escaped. Note that zero is returned to indicate a zero value, rather than the null string. Strings containing spaces or other special characters should be in quotation marks. Integer-valued arguments may be preceded by a unary minus sign. Internally, integers are treated as 32-bit numbers in two's complement form.

The operators and keywords are listed below. Expressions should be quoted by the shell, since many of the characters that have special meaning in the shell also have special meaning in *expr*. The list is in order of increasing precedence, with operators of equal precedence grouped within braces ({ }).

expr | expr

Returns the first *expr* if it is neither null nor 0, otherwise returns the second *expr*.

expr & expr

Returns the first *expr* if neither *expr* is null nor 0, otherwise returns 0.

expr { =, >, >=, <, <=, != } *expr*

Returns the result of an integer comparison if both arguments are integers, otherwise returns the result of a lexical comparison.

expr { +, - } *expr*

Addition or subtraction of integer-valued arguments.

expr { *, /, % } *expr*

Multiplication, division, or remainder of the integer-valued arguments.

expr : *expr*

The matching operator (:) compares the first argument with the second argument, which must be a regular expression; regular expression syntax is the same as that of *ed*(C), except that all patterns are “anchored” (i.e., begin with a caret (^)); hence the caret is not used as a special character. (Note that in the shell, the caret has the same meaning as the pipe symbol (|).) Normally the matching operator returns the number of characters matched (zero on failure). Alternatively, the *\(...\)* pattern symbols can be used to return a portion of the first argument.

Examples

1. `a='expr $a + 1'`

Adds 1 to the shell variable *a*.

2. `# For $a equal to either "/usr/abc/file" or just "/file"`
`'expr $a : .*\/(.*) | $a'`

Returns the last segment of a pathname (i.e., file). Watch out for the slash alone as an argument: *expr* will take it as the division operator (see “Notes”).

3. `expr $VAR : '.*'`

Returns the number of characters in \$VAR.

See Also

`ed(C)`, `sh(C)`

Diagnostics

As a side effect of expression evaluation, *expr* returns the following exit values:

0	If the expression is neither null nor zero
1	If the expression is null or zero
2	For invalid expressions

Other diagnostics include:

<i>syntax error</i>	For operator/operand errors
<i>nonnumeric argument</i>	If arithmetic is attempted on such a string

Notes

After argument processing by the shell, *expr* cannot tell the difference between an operator and an operand except by the value. If \$a is an equals sign (=), the command:

```
expr $a = =
```

looks like:

`expr = = =`

Thus the arguments are passed to *expr* (and will all be taken as the = operator). The following permits comparing equals signs:

`expr X$a = X=`



Name

factor – Factors a number.

Syntax

factor [*number*]

Description

When *factor* is invoked without an argument, it waits for a number to be typed in. If you type in a positive number less than 10^{14} , *factor* will factor the number and print its prime factors; each one is printed the proper number of times.

If *factor* is invoked with an argument, it factors the number as above and then exits. If *factor* is invoked without an argument, it factors each number supplied and then waits for another number. The *factor* command exits if it encounters a zero or any nonnumeric character.

The time it takes to factor a number, n , is proportional to $\sqrt{n}$. It usually takes longer to factor a prime or the square of a prime, than to factor other numbers.

Diagnostics

The *factor* command returns an error message if the supplied input value is greater than 10^{14} or if it is not an integer number.



Name

false – Returns with a nonzero exit value.

Syntax

false

Description

False does nothing except return with a nonzero exit value. *True*(C), *false*'s counterpart, does nothing except return with a zero exit value. False is typically used in shell procedures such as:

```
until false
do
    command
done
```

See Also

sh(C), true(C)

Diagnostics

False is any non-zero value.



Name

`file` – Determines file type.

Syntax

`file [ -m ] [ -f file ] file ...`

Description

File performs a series of tests on each argument in an attempt to classify it. If an argument appears to be ASCII, *file* examines the first 512 bytes and tries to guess its language. If a file argument is an executable `a.out`, *file* will print the version stamp if it is greater than 0 (see *ld*(CP)).

If the `-f` option is given, the next argument is considered to be a file containing the names of the files to be examined.

The `-m` option sets the access time for the examined file to the current time. Otherwise, the access time remains unchanged.

Several object file formats are recognized. For `a.out` and `x.out` format object files, the relationship of *cc* flags to file classification is `-i` for “separate”, `-n` for “pure”, and `-s` for not “not stripped”.

Notes

File can make mistakes: in particular it often suggests that command files are C programs.

Credit

This utility was developed at the University of California at Berkeley and is used with permission.



Name

find – Finds files.

Syntax

find *pathname-list expression*

Description

The *find* command recursively descends the directory hierarchy for each *pathname* in the *pathname-list* (i.e., one or more pathnames) seeking files that match a Boolean *expression* written in the following primaries. In these descriptions, the *n* argument is used as a decimal integer, where *+n* means more than *n*, *-n* means less than *n*, and *n* means exactly *n*.

- inum *n*** True if file has *n* as i-node number.
- name *file*** True if *file* matches the current filename. Normal shell argument syntax may be used if escaped (watch out for the left bracket ([), the question mark (?) and the star (*).
- perm *onum*** True if the file permission flags exactly match the octal number *onum* (see *chmod*(C)). If *onum* is prefixed by a minus sign, more flag bits (01777, see *stat*(S)) become significant.
- type *x*** True if the type of the file is *x*, where *x* is **b**, **c**, **d**, **p**, **s**, **m**, or **f** for block special files, character special files, directories, named pipes, semaphores, shared data file, or plain files, respectively.
- links *n*** True if the file has *n* links.
- user *uname*** True if the file belongs to the user, *uname*. If *uname* is numeric and does not appear as a login name in the */etc/passwd* file, it is taken as a user ID.
- group *gname*** True if the file belongs to the *gname* group. If *gname* is numeric and does not appear in the */etc/group* file, it is taken as a group ID.
- size *n*** True if the file is *n* blocks long (512 bytes per block).
- atime *n*** True if the file has been accessed in the last *n* days.
- mtime *n*** True if the file has been modified in the last *n* days.

<code>-ctime <i>n</i></code>	True if the file has been changed in the last <i>n</i> days.
<code>-exec <i>cmd</i></code>	True if the executed <i>cmd</i> returns a zero value as exit status. The end of <i>cmd</i> must be punctuated by an escaped semicolon. A command argument { } is replaced by the current pathname.
<code>-ok <i>cmd</i></code>	Like <code>-exec</code> except that the generated command line is printed with a question mark first, and is executed only if the user responds by typing “y”.
<code>-cpio <i>device</i></code>	Always true; write the current file on <i>device</i> in <i>cpio</i> (F) format (5120-byte records).
<code>-print</code>	Always true; causes the current pathname to be printed.
<code>-newer <i>file</i></code>	True if the current file has been modified more recently than the argument <i>file</i> .
<code>(<i>expression</i>)</code>	True if the expression in parentheses is true (parentheses are special to the shell and must be escaped).

The primaries may be combined using the following operators (in order of decreasing precedence):

negation	The negation of a primary is specified with the exclamation (!) unary <i>not</i> operator.
AND	The AND operation is implied by the juxtaposition of two primaries.
OR	The OR operation is specified with the <code>-o</code> operator given between two primaries.

Example

The following removes all files named `a.out` or `*.out` that have not been accessed for a week:

```
find / \( -name a.out -o -name '*.o' \) -atime +7 -exec rm {} \;
```

Files

```
/etc/passwd
/etc/group
```

Notes

The *find* command has a sub-directory level limit on searches of 20 nested sub-directories.

See Also

`cpio(C)(F)`, `sh(C)`, `stat(S)`, `test(C)`



Name

finger – Finds information about users.

Syntax

```
finger [ -bfilpqsw ] [ login1 [ login2 ... ] ]
```

Description

By default, *finger* lists the login name, full name, terminal name and write status (as a “*” before the terminal name if write permission is denied), idle time, login time, office location, and phone number (if known) for each current XENIX user. (Idle time is in minutes if it is a single integer, hours and minutes if a colon (:) is used, or days and hours if a “d” is used.)

A longer format also exists and is used by *finger* whenever a list of names is given. (Account names as well as first and last names of users are accepted.) This is a multiline format; it includes all of the information described above as well as the user’s home directory and login shell, any plan which the person has placed in the *.plan* file in their home directory, and the project on which they are working from the *.project* file, also in the home directory.

Finger options are:

- b** Prints briefer long output format of users.
- f** Suppresses the printing of the header line (short format)
- i** Prints quick list of users with idle times.
- l** Forces long output format.
- p** Suppresses printing of the *.plan* files.
- q** Prints quick list of users.
- s** Forces short output format.
- w** Forces narrow format list of specified users.

Files

/etc/utmp	Who file
/etc/passwd	User names, offices, phones, login directories, and shells
\$HOME/.plan	Plans

\$HOME/.project

Projects

See Also

who(C)

Notes

Only the first line of the `.project` file is printed.

The “office” column of the output will contain any text in the comment field of the user’s `/etc/passwd` file entry that immediately follows a comma (,). For example, if the entry is

```
johnd:eX8HinAk:201:50:John Doe, 321:/usr/johnd:/bin/sh
```

the number 321 will appear in the office column.

Idle time is computed as the elapsed time since any activity on the given terminal. This includes previous invocations of *finger* which may have modified the terminal’s corresponding device file, `/dev/tty??`.

Credit

This utility was developed at the University of California at Berkeley and is used with permission.

Name

`format` – Organizes floppy diskettes and disk cartridges into tracks and sectors.

Syntax

`format [-qy] special-file`

Description

Format formats two types of media: floppy disks and disk cartridges, based on the *specialfile* specified. By default, *format* performs a read-verify if a floppy diskette is formatted. If the `-q` option is selected and a floppy diskette is being formatted, no read-verification is performed, allowing the diskette to be formatted more quickly.

The *specialfile* must be the name of the character I/O device special file that corresponds to the floppy disk drive or disk cartridge drive.

When formatting a floppy disk, you must specify a format that is compatible with the drive in which the diskette is being formatted.

For example, the command

```
format /dev/rfd096ds15
```

formats a high-capacity diskette in the high-capacity floppy drive 0. The command

```
format /dev/rcd0
```

formats the disk cartridge in cartridge drive 0 (if installed).

Formatting a floppy disk or disk cartridge destroys the data on it. By default, *format* checks to see whether data is stored on the media before formatting it. If data is found, *format* displays the message

```
Warning: device contains data. Overwrite? (y/n):
```

If you still wish to overwrite the data, press 'Y' and press RETURN. Otherwise, press 'N' and press RETURN. This prompt is not displayed if you select the `-y` option.

Note that the floppy disk or disk cartridge must be in the appropriate drive before invoking the command.

Files

/bin/format

Notes

Formatting a floppy diskette or disk cartridge destroys any information already stored on the media.

An error message appears on the console if you run *format* on media that has never been formatted before. This is normal and the format continues.

You must place the media in the drive before executing the *format* command.

Format only formats the diskette in the drive of the system unit. Keep this in mind if you are working from a terminal.

There are two basic types of floppy diskettes you can use with your computer. They are:

High Density:

80 tracks, 15 sectors/track (single and double sided)

Non-High Density:

40 tracks, 8 sectors/track (single and double sided)

40 tracks, 9 sectors/track (single and double sided)

With a 1.2 Meg (High Capacity) drive you can format, read from, and write to a High Density diskette. You can also read from and write to a Non-High Density diskette, but you will not be able to read this diskette in any drive other than a 1.2 Meg drive.

With a 360K drive, you can format, read from, and write to a Non-High Density diskette. You cannot use a High-Density diskette.

The disk cartridge uses one track for the boot track. During the formatting procedure, information describing the size and shape of the formatted media is written to the cartridge. Each cartridge contains spare areas that are used to maintain a constant-sized area for data storage. If these extra areas are all used, then the cartridge will no longer format.

A 10-megabyte cartridge has 9760 blocks available for use by *mkfs*, *tar*, or *restore*. A 20-megabyte cartridge has 20880 blocks available.

Never remove a floppy diskette from its drive until the drive light goes out.

Name

`fsck` – Checks and repairs file systems.

Syntax

`/etc/fsck [ options ] [ file-system ] ...`

Description

Fsck audits and interactively repairs inconsistent conditions for XENIX System V or version 3.0 file systems. If the file system is consistent then the number of files, number of blocks used, and number of blocks free are reported. If the file system is inconsistent the operator is prompted for concurrence before each correction is attempted. Note that most corrective actions result in some loss of data. The amount and severity of the loss may be determined from the diagnostic output. The default action for each consistency correction is to wait for you to respond “yes” or “no”. If you do not have write permission, *fsck* defaults to the action of the `-n` option.

The following flags are interpreted by *fsck*:

- `-y` Assumes a “yes” response to all questions asked by *fsck*.
- `-n` Assumes a “no” response to all questions asked by *fsck*; do not open the file system for writing.
- `-sb:c` Ignores the actual free-list and (unconditionally) reconstructs a new one by rewriting the super-block of the file system. The file system *must* be unmounted while this is done.

The `-sb:c:c` option allows for creating an optimal free-list organization. The following forms are supported:

`-s`
`-sBlocks-per-cylinder:Blocks-to-skip (for anything else)`

If *b:c* is not given, then the values used when the file system was created are used. If these values were not specified, then a reasonable default value is used.

- `-S` Conditionally reconstructs the free-list. This option is like `-sb:c` above except that the free-list is rebuilt only if there are no discrepancies discovered in the file system. Using `-S` forces a “no” response to all questions asked by *fsck*. This option is useful for forcing free-list reorganization of uncontaminated file systems.

- t** If *fsck* cannot obtain enough memory to keep its tables, it uses a scratch file. If the **-t** option is specified, the file named in the next argument is used as the scratch file. Without the **-t** flag, *fsck* prompts the operator for the name of the scratch file. The file chosen should not be on the file system being checked, and if it is not a special file or did not already exist, it is removed when *fsck* completes.
- q** Quiet *fsck*. Do not print size-check messages in Phase 1. Unreferenced fifos will be removed selectively. If *fsck* requires it, counts in the super-block will be fixed automatically and the free-list salvaged.
- D** Directories are checked for bad blocks. This option is useful after system crashes.
- f** Fast check. Check block and sizes (Phase 1) and check the free-list (Phase 5). The free list will be reconstructed (Phase 6) if necessary.
- rr** Recovers the **root** file system. The required *filesystem* argument must refer to the **root** file system, and preferably to the block device (normally **/dev/root**). This switch implies **-y** and overrides **-n**. If any modifications to the file system are required, the system will be shutdown automatically to insure the integrity of the file system.
- c** Converts any supported file system to the type of the current file system. You are asked to verify the request for each file system that requires conversion unless you specify the **-y** option. It is recommended that every file system be checked with this option *while unmounted* if it is to be used with the current version of XENIX. To update the active **root** file system, check it with:

```
fsck -c -rr /dev/root
```

If no *filesystems* are specified, *fsck* reads a list of default file systems from the **/etc/checklist** file.

Inconsistencies checked are as follows:

- Blocks claimed by more than one inode or by the free-list
- Blocks claimed by an inode or by the free-list outside the range of the file system
- Incorrect link counts
- Size checks:
 - Incorrect number of blocks
 - Directory size not 16-byte aligned

- Bad inode format
- Blocks not accounted for anywhere
- Directory checks:
File pointing to unallocated inode
Inode number out of range
- Super-block checks:
More than 65536 inodes
More blocks for inodes than there are in the file system
- Bad free block list format
- Total free block or free inode count incorrect

Orphaned files and directories (allocated but unreferenced) are, with the operator's concurrence, reconnected by placing them in the **lost+found** directory. The name assigned is the inode number. The only restriction is that the **lost+found** directory must exist in the root of the file system being checked and must have empty slots in which entries can be made. You do this by making a **lost+found** directory, copying a number of files to the directory, and then removing them (before *fsck* is executed).

Files

`/etc/checklist` Contains default list of file systems to check

See Also

`checklist(F)`, `filesystem(F)`

Diagnostics

The diagnostics produced by *fsck* are self-explanatory.

Notes

The *fsck* command cannot handle sub-directories greater than 34 levels deep.

Fsck will not run on a *mounted* file system unless the file system is the **root** file system, or the `-n` option is specified and no writing out of the file system will take place. If any such attempt is made, a warning is printed and no further processing of the file system is done for the specified device.

Although checking a raw device is almost always faster, there is no way to tell if the file system is mounted. And cleaning a mounted file system will almost certainly result in an inconsistent super-block.

Warning

File systems created under XENIX version 2.3 are not supported under XENIX version 3.0 because the word ordering in type *long* variables has been changed. *Fsck* is capable of auditing and repairing XENIX version 2.3 file systems if the word ordering is correct. Therefore, if you have a version 2.3 file system that must be used, swap the appropriate words and then run *fsck*. After this is done, the version 2.3 file system can be mounted under a XENIX version 3.0 kernel.

For the **root** file system, run “*fsck -rr /dev/root*”, and for all other file systems, use “*fsck /dev/??*” on the *unmounted* block device.

Name

`getopt` – Parses command options.

Syntax

`set — `getopt optstring $*``

Description

Getopt is used to check and break up options in command lines for parsing by shell procedures. *Optstring* is a string of recognized option letters (see *getopt(S)*). If a letter is followed by a colon, the option is expected to have an argument which may or may not be separated from it by white space. The special option `—` is used to delimit the end of the options. *Getopt* will place `—` in the arguments at the end of the options, or recognize it if it is used explicitly. The shell arguments (`$1 $2 . . .`) are reset so that each option is preceded by a dash (`-`); each option argument is also in its own shell argument.

Example

The following code fragment shows you how to process the arguments for a command that can take the `-a` and `-b` options, and the `-o` option, which requires an argument:

```
set -- `getopt abo: $*`
if [ $? != 0 ]
then
    echo $USAGE
    exit 2
fi
for i in $*
do
    case $i in
        -a | -b)    FLAG=$i; shift;;
        -o)        OARG=$2;    shift; shift;;
        - -)       shift;    break;;
    esac
done
```

This code will accept any of the following as equivalent:

```
cmd -aoarg file file
cmd -a -o arg file file
cmd -oarg -a file file
cmd -a -oarg — file file
```

See Also

getopt(S), sh(C)

Diagnostics

Getopt prints an error message on the standard output when it encounters an option letter not included in *optstring*.

Name

grep, egrep, fgrep – Searches a file for a pattern.

Syntax

grep [*options*] *expression* [*files*]

egrep [*options*] [*expression*] [*files*]

fgrep [*options*] [*strings*] [*files*]

Description

Commands of the *grep* family search the input *files* (standard input default) for lines matching a pattern. Normally, each line found is copied to the standard output. *Grep* patterns are limited regular *expressions* in the style of *ed*(C); it uses a compact nondeterministic algorithm. *Egrep*, whose patterns are full regular *expressions*; it uses a fast deterministic algorithm that sometimes needs exponential space. *Fgrep* patterns are fixed *strings*; it is fast and compact. The following *options* are recognized:

- v All lines but those matching are printed.
- x Prints only exact matches of an entire line. (*Fgrep* only.)
- c Only a count of matching lines is printed.
- l Only the names of files with matching lines are listed, separated by NEW-LINES.
- h Prevents the name of the file containing the matching line from being appended to that line. Use when searching multiple files.
- n Each line is preceded by its relative line number in the file.
- b Each line is preceded by the block number on which it was found. This is sometimes useful in locating disk block numbers by context.
- s Suppresses error messages produced for nonexistent or unreadable files. (*grep* only)
- y Turns on matching of letters of either case in the input so that case is insignificant. This does not work for *egrep*.
- e *expression*
Same as a simple *expression* argument, but useful when the *expression* begins with a dash (–).

-f file

The regular *expression* for *grep* or *egrep*, or *strings* list (for *fgrep*) is taken from the *file*.

In all cases, the filename is output if there is more than one input file. Be careful when using the \$, *, [, ^, |, (,), and \ characters in *expression*, because they are also meaningful to the shell. It is safest to enclose the entire *expression* argument in single quotation marks.

Fgrep searches for lines that contain one of the *strings* separated by NEWLINEs.

Egrep accepts regular expressions as in *ed*(C), except for \ (and \), with the addition of the following:

- A regular expression followed by a plus sign (+) matches one or more occurrences of the regular expression.
- A regular expression followed by a question mark (?) matches zero or one occurrences of the regular expression.
- Two regular expressions separated by a vertical bar (|) or NEWLINE match strings that are matched by either regular expression.
- A regular expression may be enclosed in parentheses () for grouping.

The order of precedence of operators is [], *?+, then concatenation, then the backslash (\) and the NEWLINE.

See Also

ed(C), *sed*(C), *sh*(C)

Diagnostics

Exit status is zero (0) if any matches are found, one (1) if none, two (2) for syntax errors or inaccessible files.

Notes

Ideally, there should be only one *grep*, but there is no single algorithm that is both fast and efficient.

Lines are limited to 256 characters; longer lines are truncated.

Egrep does not recognize ranges, such as [a-z], in character classes.

When *grep* is used with the *-y* option, the search is not made totally case insensitive in character ranges specified within brackets.

Multiple strings can be specified in *fgrep* without using a separate strings file by using the quoting conventions of the shell to embed NEWLINES in the *single* string argument. For example, you might type the following at the command line:

```
fgrep 'string1
string2
string3' text.file
```

Similarly, multiple strings can be specified in *egrep* by doing:

```
egrep 'string1|string2|string3' text.file
```

Thus, *egrep* can do almost anything that *grep* and *fgrep* can do.



Name

`grpcheck`, `grpck` – Checks group file.

Syntax

`grpcheck` [*file*]

`grpck` [*file*]

Description

Grpcheck verifies all entries in the group file. *Grpcheck* and *grpck* are equivalent commands and may be used interchangeably. This verification includes a check of the number of fields, group name, group ID, and whether all login names appear in the password file. The default group file is `/etc/group`.

Files

`/etc/group`

`/etc/passwd`

See Also

`group(M)`, `passwd(M)`, `pwcheck(C)`

Diagnostics

Group entries in `/etc/group` with no login names are flagged.

Name

haltsys – Closes out the file systems and halts the CPU.

Syntax

/etc/haltsys

Description

Haltsys does a *shutdn()* system call (see *shutdn(S)*) to flush out pending disk I/O, mark the file systems clean, and halt the processor. *Haltsys* takes effect immediately, so user processes should be killed beforehand. *Shutdown(C)* is recommended for normal system termination; it warns the users, cleans things up, and calls *haltsys*. Use *haltsys* directly only if some system problem prevents the running of *shutdown*.

See Also

shutdn(S), shutdown(C)



Name

`hd` – Displays files in hexadecimal format.

Syntax

`hd [ -format ... ] [ -s offset ] [ -n count ] [ file ] ...`

Description

The `hd` command displays the contents of files in hexadecimal, octal, decimal, and character formats. Control over the specification of ranges of characters is also available. The default behavior is with the following flags set: “`-abx -A`”. This says that addresses (file offsets) and bytes are printed in hexadecimal and that characters are also printed. If no *file* argument is given, the standard input is read.

Options include:

`-s offset` Specify the beginning offset in the file where printing is to begin. If no ‘*file*’ argument is given, or if a seek fails because the input is a pipe, ‘*offset*’ bytes are read from the input and discarded. Otherwise, a seek error will terminate processing of the current file.

The *offset* may be given in decimal, hexadecimal (preceded by ‘`0x`’), or octal (preceded by a ‘`0`’). It is optionally followed by one of the following multipliers: `w`, `l`, `b`, or `k`; for words (2 bytes), long words (4 bytes), half kilobytes (512 bytes), or K bytes (1024 bytes). Note that this is the one case where ‘`b`’ does *not* stand for bytes. Since specifying a hexadecimal offset in blocks would result in an ambiguous trailing ‘`b`’, any offset and multiplier may be separated by an asterisk (*).

`-n count` Specify the number of bytes to process. The *count* is in the same format as *offset*, above.

Format Flags

Format flags may specify addresses, characters, bytes, words (2 bytes), or longs (4 bytes) to be printed in hexadecimal, decimal, or octal. Two special formats may also be indicated: text or ASCII. Format and base specifiers may be freely combined and repeated as desired in order to specify different bases (hexadecimal, decimal or octal) for different output formats (addresses, characters, etc.). All format flags appearing in a single argument are applied as appropriate to all other flags in that argument.

acbwIA

Output format specifiers for addresses, characters, bytes, words, longs and ASCII, respectively. Only one base specifier will be used for addresses; the address will appear on the first line of output that begins each new offset in the input.

The character format prints printable characters unchanged, special C escapes as defined in the language, and remaining values in the specified base.

The ASCII format prints all printable characters unchanged, and all others as a period (.). This format appears to the right of the first of other specified output formats. A base specifier has no meaning with the ASCII format. If no other output format (other than addresses) is given, **bx** is assumed. If no base specifier is given, *all* of **xdo** are used.

xdo

Output base specifiers for hexadecimal, decimal and octal. If no format specifier is given, *all* of **acbwI** are used.

- t** Print a text file, each line preceded by the address in the file. Normally, lines should be terminated by a `\n` character; but long lines will be broken up. Control characters in the range 0x00 to 0x1f are printed as '^@' to '^_'. Bytes with the high bit set are preceded by a tilde (~) and printed as if the high bit were not set. The special characters (^, ~, \) are preceded by a backslash (\) to escape their special meaning. As special cases, two values are represented numerically as '\177' and '\377'. This flag will override all output format specifiers except addresses.

Name

hdinit – Creates a bootable XENIX system on a fixed disk.

Syntax

hdinit [**-s**]

Description

The *hdinit* command is an interactive shell script used to install the XENIX operating system and a minimal set of utilities on a fixed disk. Only the system administrator should use this command, after having booted XENIX from the Installation diskette.

The *hdinit* command presents a series of menus, from which the system administrator chooses how to partition the fixed disk and defines the value of installation dependent constants such as the time zone.

You can also use the *hdinit* command to change the size of the swap area in the root partition by invoking the **-s** option. See Chapter 5 of the *XENIX Installation Guide* for more detailed information on this option.

Files

/etc/hdinit

See Also

badtrack(C), fdisk(C), xinstall(C)

Diagnostics

The *hdinit* command refuses to run if it cannot find a badtrack table on the fixed disk. In this case, you should reboot the system, and run /etc/badtrack.

You must format the fixed disk before running *hdinit*. See your computers operations manual for instructions.

See Also

XENIX Installation Guide



Name

`head` – Prints the first few lines of a stream.



Syntax

`head [ -count ] [ files ... ]`

Description

This filter prints the first *count* lines of each of the specified *files*. If no *files* are specified, *head* reads from the standard input. If no *count* is specified, then 10 lines are printed.

See Also

`tail(C)`

Credit



This utility was developed at the University of California at Berkeley and is used with permission.





Name

hello – Write to another user.

Syntax

hello *user* [*ttyname*]

Description

Hello copies lines from your terminal to that of another user. When first called, *hello* sends the message: Hello from *yourname yourttyname*... The recipient of the message should write back at this point. Communication continues until an end of file is read from the terminal or an interrupt is sent. At that point *hello* writes “bye now” on the other terminal and exits.

If you want to write to a user who is logged at more than one terminal, use the *ttyname* argument to indicate the appropriate terminal name.

Permission to write may be denied or granted by use of the *mesg* command which by default allows writing. Certain commands, such as *nroff* and *pr* disallow messages in order to prevent messy output.

If you try to write to a user who has messages disabled, *hello* sends mail to the user saying that a *hello* was attempted by you. This also happens if the user is not currently logged in. The following protocol is suggested for using *hello* : when you first write to another user, wait for him to write back before starting to send a message. Each party should end their message with a distinctive signal, such as “o” for “over” so the other knows to reply. Another suggestion is the use of “oo” for “over and out” when conversation is about to be terminated.

Files

/etc/utmp
/bin/sh

See Also

mesg(C), who(C), mail(C)



Name

`help` – Locates a manual page for a command.

Syntax

`help [ options ] [ subject ]`

Description

Help is a menu driven, interactive command to aid you in locating manual pages for other commands.

You may include the following options on the command line:

`-s section`

Specifies a manual section. If you omit `-s`, *help* displays all occurrences of the command in all sections of the manual.

`-k keyword`

Displays a brief description of *keyword*.

`-l` Lists available *help* topics organized by manual section.

`-d database`

Specifies an alternate *help* database, allowing you to create your own database for the *help* command.

If you include a command name on the command line, *help* displays the manual page without loading the database.

While loading the database, *help* displays one period (.) for every fifty lines in the database. After loading the database, *help* displays a menu of options. Select the description that is appropriate for the command you wish to see. Depending on your selection, *help* may display a number of menus before displaying the manual page for the command.

Files

`/usr/lib/Help/helplist` - the default help database

Diagnostics

Database line number n has a tab in it.

Tabs are not permitted in the database file.

Database line number n has an odd number of spaces.

All lines in the database file must be preceded by an even number of spaces.
See *helplist(F)* for more information on the database layout.

Out of memory

Insufficient memory to initialize the database.

See Also

helplist(F)

Credit

Robert K. Ware, University of Denver
Tandy Systems Software

Name

id – Prints user and group IDs and names.

 Syntax

id

Description

Id writes a message on the standard output giving the user and group IDs and the corresponding names of the invoking process. If the effective and real IDs do not match, both are printed.

See Also

getuid(S), logname(C)



Name

install – Install commands.

Syntax

```
install [-c dira] [-f dirb] [-i] [-n dirc] [-o] [-s] file [dirx ...]
```

Description

Install is a command most commonly used in “makefiles” (see *make*(CP)) to install a *file* (updated target file) in a specific place within a file system. Each *file* is installed by copying it into the appropriate directory, thereby retaining the mode and owner of the original command file. The program prints messages telling you exactly what files it is replacing or creating and where they are going.

If no options or directories (*dirx* ...) are given, *install* will search (using *find*(C)) a set of default directories (*/bin*, */usr/bin*, */etc*, */lib*, and */usr/lib*, in that order) for a file with the same name as *file*. When the first occurrence is found, *install* issues a message saying that it is overwriting that file with *file*, and proceeds to do so. If the file is not found, the program states this and exits without further action.

If one or more directories (*dirx* ...) are specified after *file*, those directories will be searched before the directories specified in the default list.

The meanings of the options are:

- c *dira*** Installs a new command file in the directory specified in *dira*. Looks for *file* in *dira* and installs it there if it is not found. If it is found, *install* issues a message saying that the file already exists, and exits without overwriting it. May be used alone or with the **-s** option.
- f *dirb*** Forces *file* to be installed in given directory, whether or not one already exists. If the file being installed does not already exist, the mode and owner of the new file will be set to **755** and **bin**, respectively. If the file exists, the mode and owner will be that of the existing file. May be used alone or with the **-o** or **-s** options.
- i** Ignores default directory list, searching only through the given directories (*dirx* ...). May be used alone or with any other options except **-c** and **-f**.
- n *dirc*** If *file* is not found in any of the searched directories, it is put in the directory specified in *dirc*. The mode and owner of the new file will be set to **755** and **bin**, respectively. May be used alone or with any other options except **-c** and **-f**.

- o** If *file* is found, this option saves the “found” file by copying it to *oldfile* in the directory in which it was found. May be used alone or with any other options except **-c**.
- s** Suppresses printing of messages other than error messages. May be used alone or with any other options.

See Also

find(C), make(CP)

Name

`ipcrm` – Removes a message queue, semaphore set or shared memory ID.

Syntax

`ipcrm [ options ]`

Description

Ipcrm will remove one or more specified messages, semaphore or shared memory identifiers. The identifiers are specified by the following *options*:

- q *msqid*** Removes the message queue identifier, *msqid*, from the system and destroys the message queue and data structure associated with it.
- m *shmid*** Removes the shared memory identifier, *shmid*, from the system. The shared memory segment and data structure associated with it are destroyed after the last detach.
- s *semid*** Removes the semaphore identifier, *semid*, from the system and destroys the set of semaphores and data structure associated with it.
- Q *msgkey*** Removes the message queue identifier, created with *msgkey*, from the system and destroys the message queue and data structure associated with it.
- M *shmkey*** Removes the shared memory identifier, created with *shmkey*, from the system. The shared memory segment and data structure associated with it are destroyed after the last detach.
- S *semkey*** Removes the semaphore identifier, created with *semkey*, from the system and destroys the set of semaphores and data structure associated with it.

The details of the removes are described in *msgctl(S)*, *shmctl(S)*, and *semctl(S)*. The identifiers and keys may be found by using *ipcs(C)*.

See Also

ipcs(C), *msgctl(S)*, *msgget(S)*, *msgop(S)*, *semctl(S)*, *semget(S)*, *semop(S)*, *shmctl(S)*, *shmget(S)*, *shmop(S)*

Note

You cannot use *ipcrm* to remove semaphores which were created using *creatsem(S)* or to remove shared memory which was created using *sdget(S)*.

Name

`ipcs` – Reports the status of interprocess communication facilities.

Syntax

`ipcs [ options ]`

Description

Ipcs prints certain information about active interprocess communication facilities. Without *options*, information is printed in short format for message queues, shared memory, and semaphores that are currently active in the system. Otherwise, the displayed information is controlled by the following *options*:

- `-q` Print information about active message queues.
- `-m` Print information about active shared memory segments.
- `-s` Print information about active semaphores.

If any of the `-q`, `-m`, or `-s` options are specified, *ipcs* will print only information about the indicated options.

- `-b` Print largest allowable size information. (Maximum number of bytes in messages on queue for message queues, size of segments for shared memory, and number of semaphores in each set for semaphores.) See below for the meaning of columns in a listing.
- `-c` Print creator's login name and group name.
- `-o` Print information on outstanding usage. (Number of messages on queue and total number of bytes in messages on queue for message queues and number of processes attached to shared memory segments.)
- `-p` Print process number information. (ID of the last process to send a message, ID of the last process to receive a message on the message queues, ID of the creating process, and ID of the last process to attach or detach on shared memory segments)
- `-t` Print time information. (Time of the last control operation that changed the access permissions for all facilities. Time of last *msgsnd* and last *msgrcv* on message queues, last *shmat* and last *shmdt* on shared memory, last *semop*(S) on semaphores.)
- `-a` Use all print *options*. (This is a shorthand notation for the `-b`, `-c`, `-o`, `-p`, and `-t` options.)

- C *corefile*** Use the **corefile** file in place of **/dev/kmem**.
- N *namelist*** The argument will be taken as the name of an alternate *namelist* (**/xenix** is the default).

The column headings and the meaning of the columns in an *ipcs* listing are given below; the letters in parentheses indicate the *options* that cause the corresponding heading to appear; *All* means that the heading always appears. Note that these *options* only determine what information is provided for each facility; they do *not* determine which facilities will be listed.

- T** (all)
Type of the facility:
- | | |
|----------|------------------------|
| q | Message queue; |
| m | Shared memory segment; |
| s | Semaphore. |
- ID** (all)
The identifier for the facility entry. Note that ID is "X" for facilities created using *creatsem(S)* or *sdget(S)*.
- KEY** (all)
The key used as an argument to *msgget*, *semget*, or *shmget* to create the facility entry. (Note: The key of a shared memory segment is changed to **IPC_PRIVATE** when the segment has been removed until all processes attached to the segment detach it. The key is "XENIX" for facilities created using *creatsem(S)* or *sdget(S)*.)
- MODE** (all)
The facility access modes and flags: The mode consists of 11 characters that are interpreted as follows:
- The first two characters are:
- | | |
|----------|--|
| R | If a process is waiting on a <i>msgrcv</i> ; |
| S | If a process is waiting on a <i>msgsnd</i> ; |
| D | If the associated shared memory segment has been removed. It will disappear when the last process attached to the segment detaches it; |
| C | If the associated shared memory segment is to be cleared when the first attach is executed; |
| - | If the corresponding special flag is not set. |
- The next nine characters are interpreted as three sets of three bits each. The first set refers to the owner's permissions; the next to permissions of others in the user-group of the facility entry; and the last to all others. Within each set, the first character indicates permission to read, the second character indicates permission to write or alter the facility entry, and the last character is currently unused.

The permissions are indicated as follows:

r If read permission *is* granted;
w If write permission *is* granted;
a If alter permission *is* granted;
- If the indicated permission is *not* granted.

OWNER	(all)	The login name of the owner of the facility entry.
GROUP	(all)	The group name of the group of the owner of the facility entry.
CREATOR	(a,c)	The login name of the creator of the facility entry.
CGROUP	(a,c)	The group name of the group of the creator of the facility entry.
CBYTES	(a,o)	The number of bytes in messages currently outstanding on the associated message queue.
QNUM	(a,o)	The number of messages currently outstanding on the associated message queue.
QBYTES	(a,b)	The maximum number of bytes allowed in messages outstanding on the associated message queue.
LSPID	(a,p)	The ID of the last process to send a message to the associated queue.
LRPID	(a,p)	The ID of the last process to receive a message from the associated queue.
STIME	(a,t)	The time the last message was sent to the associated queue.
RTIME	(a,t)	The time the last message was received from the associated queue.
CTIME	(a,t)	The time when the associated entry was created or changed.
NATTCH	(a,o)	The number of processes attached to the associated shared memory segment.
SEGSZ	(a,b)	The size of the associated shared memory segment.
CPID	(a,p)	The process ID of the creator of the shared memory entry.
LPID	(a,p)	The ID of the last process to attach or detach the shared memory segment.
ATIME	(a,t)	The time the last attach was completed to the associated shared memory segment.
DTIME	(a,t)	The time the last detach was completed on the associated shared memory segment.
NSEMS	(a,b)	The number of semaphores in the set associated with the semaphore entry.
OTIME	(a,t)	The time the last semaphore operation was completed on the set associated with the semaphore entry.

Files

/xenix	System namelist
/dev/kmem	Memory
/etc/passwd	User names
/etc/group	Group names

See Also

`creatsem(S)`, `msgop(S)`, `sdget(S)`, `semop(S)`, `shmop(S)`

Notes

Things can change while *ipcs* is running; the picture it gives is only a close approximation to reality.

Name

join – Joins two relations.

Syntax

```
join [ options ] file1 file2
```

Description

Join forms, on the standard output, a join of the two relations specified by the lines of *file1* and *file2*. If *file1* is a dash (–), the standard input is used.

Before *join* can be performed, *file1* and *file2* must be sorted in increasing ASCII collating sequence based on the fields which are to be joined, usually the first in each line.

There is one line in the output for each pair of lines in *file1* and *file2* that have identical join fields. The output line normally consists of the common field, then the rest of the line from *file1*, then the rest of the line from *file2*.

Fields are normally separated by a SPACEBAR, TAB or NEWLINE. In this case, multiple separators count as one, and leading separators are discarded.

These options are recognized:

- an* In addition to the normal output, produces a line for each unpairable line in file *n*, where *n* is 1 or 2.
- e s* Replaces empty output fields by string *s*.
- jn m* Joins on the *m*th field of file *n*. If *n* is missing, uses the *m*th field in each file.
- o list* Each output line comprises the fields specified in *list*, each element of which has the form *n.m*, where *n* is a file number and *m* is a field number.
- tc* Uses *c* character as a separator (TAB character). Every appearance of *c* in a line is significant.

See Also

awk(C), comm(C), sort(C)

Notes

With default field separation, the collating sequence is *sort -b*; with *-t*, the sequence is a plain sort.